29. Check for Balanced Parenthesis in Java — Quick Notes

Based on the uploaded transcript.

Problem Statement

Given a string containing only: ( ) { } [ ]

Determine whether the parentheses are **balanced**.

What are Balanced Parentheses?

A string is balanced if:

1. Latest opened bracket closes first (LIFO)

Correct: { [( )] }

Latest opened '(' closes first.

Incorrect: ([ )

'[' opened later but ')' closes first.

---

2. Number of opening and closing brackets should match

Correct: (())

2 opening
2 closing

Incorrect: (((

3 opening
2 closing

Examples

Input	Output
`"()"`	`true`
`"({[]})"`	`true`
`"([)]"`	`false`
`"((("`	`false`
`"())"`	`false`

Key Observation

Latest opened bracket
must close first.

This is: LIFO (Last In First Out)

Hence, use: STACK

Algorithm

Traverse each character

If Opening Bracket

Push into stack.

(
[
{
↓
stack.push(ch);

If Closing Bracket

Case 1: Stack Empty

```
if(stack.isEmpty())
   return false;
```

Means:
More closing brackets
than opening brackets.

Example: ())

Case 2: Top Doesn't Match

```
if(!isMatching(stack.peek(), ch))
   return false;
```

Example: ([)]

Here:
Top = '['
Current = ')'

Not matching.

Case 3: Matching Pair

```
stack.pop();
```

Pair processed successfully.

Final Check

After traversal: return stack.isEmpty();

Why Final isEmpty() Check?

Example:

(((

Processing:

Push (
Push (
Push (

Pop (
Pop (

Stack:

(

still remains.

Means:

More opening brackets
than closing brackets.

Therefore: return false;

Matching Function

```
static boolean isMatching(char open,
                          char close){

   return (open=='(' && close==')') ||
          (open=='{' && close=='}') ||
          (open=='[' && close==']');
}
```

Implementation

```
import java.util.ArrayDeque;
import java.util.Deque;

class Solution {

   static boolean isBalanced(String str){
```

```
        Deque<Character> stack = new ArrayDeque<>();

        for(int i=0;i<str.length();i++){

            char ch = str.charAt(i);

            if(ch=='(' || ch=='{' || ch=='['){

                stack.push(ch);
            }
            else{
                if(stack.isEmpty())
                    return false;

                if(!isMatching(stack.peek(), ch))
                    return false;
                stack.pop();
            }
        }
        return stack.isEmpty();
    }

    static boolean isMatching(char open, char close){
        return (open=='(' && close==')') ||
               (open=='{' && close=='}') ||
               (open=='[' && close==']');
    }
}
```

Dry Run

Input

({[]})

Step 1

(

Stack: (

Step 2

{

Stack: { (

Step 3

[

Stack: [{ (

Step 4

]

Top: [

Matching: YES

Pop.

Stack: { (

Step 5

}

Top: {

Matching: YES

Pop.

Stack: (

Step 6

)

Top:
(

Matching: YES

Pop.

Stack: EMPTY

Return: true

Unbalanced Example

Input: ([)]

Processing:

Push (
Push [

Current =)

Top = [
Matching? NO

Return: false

Why Use ArrayDeque?

Instead of:

```
Stack<Character> stack =
      new Stack<>();
```

Use:

```
Deque<Character> stack =
      new ArrayDeque<>();
```

Reasons

- Stack is a **legacy class**
- Extends **Vector**
- Thread-safe (synchronized)
- Extra overhead

ArrayDeque:

Faster
Non-legacy
Preferred implementation

Complexity Analysis

Time Complexity

Θ(n)
Reason: Traverse string once.

Each operation:
push()
pop()
peek()
isEmpty()

Is: O(1)

Auxiliary Space

Worst Case: (((((((

All opening brackets.

Stack stores: n elements

Therefore: Θ(n)

Important Interview Points

- Balanced Parenthesis is a **classic Stack problem**.
- Based on **LIFO** principle.
- Push **opening brackets**.
- Match **closing brackets** with stack top.
- Check **stack.isEmpty()** before popping.
- Final stack must be **empty**.
- Prefer **ArrayDeque** over **Stack**.

Quick Revision

Concept	Remember
Data Structure	Stack
Why Stack?	LIFO
Opening Bracket	Push
Closing Bracket	Match + Pop
Empty Stack on Closing	Invalid
Non-empty Stack at End	Invalid
Time Complexity	Θ(n)
Auxiliary Space	Θ(n)
Preferred Implementation	`ArrayDeque`
Legacy Implementation	`Stack`